



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения и стандартизации информационных
технологий
ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 2
по дисциплине

«Тестирование и верификация программного обеспечения»

Выполнили студенты группы ИКБО-74-23 Команда *ЗавозWW*

Кузьмин И.В.

Васильев А.А.

Тыцкий Б.С.

Мамаева К.С.

Принял

Ильичев Г.П.

Практическая

«17» октября 2025 г.

работа выполнена

«Зачтено»

«__» _____ 2025 г.

Москва 2025

Техническое задание проекта «Конвертер единиц»

1. Аннотация

Краткое описание: Программа «Конвертер единиц» разработана в учебных целях для демонстрации процесса модульного тестирования и взаимодействия ролей «Разработчик» и «Тестировщик». В отчете приведены цель, задачи, архитектура, процесс тестирования и результаты работы.

2. Введение

- Основание для выполнения работы: практическое задание по дисциплине «Программная инженерия».
- Цель работы: закрепить навыки разработки и тестирования ПО.
- Область применения: учебный процесс, демонстрация методологии тестирования.

3. Техническое задание

3.1 Цель проекта

Создание простого учебного приложения для демонстрации процесса **модульного тестирования** и взаимодействия ролей «Разработчик» и «Тестировщик».

3.2 Задачи

- Разработать программный модуль с минимум 5 функциями.
- Встроить преднамеренную ошибку в одну из функций.
- Провести цикл «разработка → тестирование → исправление → повторное тестирование».

3.3 Требования к функционалу

Приложение «Конвертер единиц» выполняет следующие операции:

1. Перевод температуры из Цельсия в Фаренгейты.
2. Перевод температуры из Фаренгейтов в Цельсии.
3. Перевод метров в километры.
4. Перевод килограммов в граммы.
5. Перевод граммов в килограммы.

3.4 Архитектура

- **Приложение** app.ru – содержит продукт
- **Документация** Документация – описание функций, инструкции по запуску и тестированию.

3.5 Процесс тестирования

1. Тестировщик получает модуль и документацию.
2. Пишет тесты для проверки функций.
3. Находит ошибку и оформляет отчет о дефекте.
4. Разработчик исправляет баг.

5. Модуль проходит повторное тестирование.

4. Ход выполнения работы

- Разработка модуля app.py.
- Запуск тестов, фиксация ошибки.
- Исправление функции.
- Повторное тестирование.

5. Результаты

- Рабочее приложение с консольным интерфейсом.
- Набор тестов, демонстрирующих процесс поиска и исправления ошибок.
- Документация

6. Выводы

- Отработан цикл разработки и тестирования.
- Продемонстрировано взаимодействие ролей «Разработчик» и «Тестирующий».
- Получен практический опыт в модульном тестировании.

7. Программа

Исходный код программы находится под рисунком 1 в 10 пункте приложения

8. Модульное тестирование программного продукта другого участника команды

8.1 Код программы

Исходные коды программы находящиеся в пункте 10 Приложение в рисунках 2 и 3

8.2 методология и описание тестов для тестирования

1. Подход к тестированию

- Использовалась методология модульного тестирования (unit testing), при которой каждая функция программы проверяется изолированно.
- Для оценки качества тестов применялся принцип мутационного тестирования: намеренно закладывались ошибки (например, неверная граница в `get_bmi_category` или отсутствие проверки веса = 0), чтобы убедиться, что тесты их выявляют.

2. Инструменты

- Язык программирования: Python 3.11.
- Фреймворк для тестирования: `pytest` (позволяет автоматически находить тесты по шаблону `test_*.py` и формировать отчёты).

3. Организация тестов

- Каждый модуль программы (`calculate_bmi`, `get_bmi_category`, `calculate_ideal_weight`, `compare_bmi`) имеет отдельный набор тестов.
- Тесты сгруппированы по функциональности: расчёт, классификация, идеальный вес, сравнение.
- Для функций, работающих с файлами (`save_results_to_file`, `load_previous_results`), тестирование проводилось с использованием временных файлов, чтобы не затрагивать реальные данные.

4. Сценарии тестирования

- Проверялись нормальные случаи (корректные входные данные).
- Проверялись граничные значения (например, BMI = 18.5, 24.9, 25).
- Проверялись ошибочные случаи (рост = 0, вес = 0 или отрицательный).
- Добавлены тесты, которые должны намеренно «падать», чтобы показать наличие ошибок в коде.

5. Критерии успешности

- Все тесты должны выполняться успешно, кроме тех, что специально направлены на выявление ошибок.
- Ошибочные тесты подтверждают, что тестовый набор способен обнаруживать дефекты.

6. Описание тестов

Для проверки корректности работы функций были разработаны юнит-тесты на языке **Python** с использованием библиотеки **pytest**. Тесты охватывают следующие сценарии:

Тесты для `calculate_bmi(weight, height_cm)`

- `test_calculate_bmi_normal` - проверка корректного расчета BMI при нормальных значениях (70 кг, 175 см).
- `test_calculate_bmi_zero_height` - проверка обработки нулевого роста (ожидается None).
- `test_calculate_bmi_negative_weight` - проверка корректности вычислений при отрицательном весе (математически допустимо).
- `test_calculate_bmi_zero_weight` - проверка обработки нулевого веса (ожидается ошибка/None).

Тесты для `get_bmi_category(bmi)`

- `test_get_bmi_category_underweight` - категория «Недостаточный вес» при $BMI < 18.5$.
- `test_get_bmi_category_normal` - категория «Нормальный вес» при BMI в пределах нормы.
- `test_get_bmi_category_obesity` - категория «Ожирение» при $BMI \geq 30$.
- `test_get_bmi_category_none` - обработка случая, когда BMI не вычислен (None).

Тесты для `calculate_ideal_weight(height_cm, target_bmi=22)`

- `test_calculate_ideal_weight_default` - расчет идеального веса при стандартном значении $BMI = 22$.

- `test_calculate_ideal_weight_custom_bmi` - расчёт идеального веса при пользовательском значении BMI (например, 25).

Тесты для `compare_bmi(bmi1, bmi2)`

- `test_compare_bmi_no_change` - сообщение о неизменности при равных значениях BMI.
- `test_compare_bmi_with_none` - корректная обработка случая, когда одно из значений равно None

8.3 Юнит-тесты

Код тестирование юнит-тестами и логи находиться в пункте 10 приложения на рисунках 4 и 5 соответственно

8.4 Анализ покрытия кода и исправление ошибок

1. Покрытие кода

- `calculate_bmi`: проверены корректные значения, нулевой рост, отрицательный вес, нулевой вес.
- `get_bmi_category`: протестированы все категории, случай `None`, а также граничные значения.
- `calculate_ideal_weight`: проверены расчеты при стандартном и пользовательском значении BMI.
- `compare_bmi`: протестированы все варианты сравнения (увеличение, уменьшение, равенство, ошибка при `None`).
- Функции работы с файлами пока не покрыты тестами, но могут быть протестированы с использованием временных файлов.

2. Обнаруженные ошибки

Краткое описание ошибки: Некорректная обработка нулевого веса.

Статус ошибки: Открыта (Open).

Категория ошибки: Серьезная (Major).

Тестовый случай: «Проверка алгоритма функционирования программы при вводе веса = 0».

Описание ошибки:

1. Загрузить программу.
2. В поле ввода веса ввести значение 0.
3. Нажать кнопку «Пуск».
4. Полученный результат: программа возвращает числовое значение BMI (result).
5. Ожидаемый результат: сообщение об ошибке — «Ошибка: вес не может быть отрицательным!».

3. Итог

- После исправлений все тесты проходят успешно.
- Тесты подтвердили корректность вычислений и классификации.

9. Анализ и выводы по модульному тестированию (BMI-калькулятор)

Оценка качества тестирования

- Для функций `calculate_bmi`, `get_bmi_category`, `calculate_ideal_weight`, `compare_bmi` были написаны отдельные тесты.
- Тесты охватывают:
 - корректный расчет BMI при нормальных значениях;
 - обработку нулевого роста (ожидается `None`);
 - проверку отрицательного веса (возврат ошибки или отрицательного значения);
 - определение категорий BMI (недостаточный вес, нормальный вес, избыточный вес, ожирение);
 - обработку ошибок (`None` и некорректные данные);
 - сравнение BMI (увеличение, уменьшение, отсутствие изменений).
- Таким образом, тесты покрывают как нормальные сценарии, так и граничные случаи.

Анализ недостатков и их устранение

- В процессе тестирования выявлено несоответствие: при весе = 0 функция `calculate_bmi` возвращала 0.0, тогда как по логике задачи должен возвращаться текст ошибки.
- Для устранения добавлена проверка `if weight <= 0: return "Ошибка: вес не может быть отрицательным!"`.
- В `get_bmi_category` добавлена обработка строковых ошибок, чтобы корректно пробрасывать сообщения об ошибках.
- После исправлений тесты были перезапущены, и все проверки прошли успешно.

Итоговые выводы

- Модульное тестирование подтвердило эффективность метода: ошибка в логике функции была обнаружена и устранена.
- Качество тестирования можно оценить как достаточное: покрыты все основные функции и ключевые сценарии.
- Для повышения качества можно добавить:
 - тесты на граничные значения категорий (18.5, 24.9, 25.0, 29.9);
 - проверки типов возвращаемых значений (float, str, None);
 - стресс-тесты на большие значения веса и роста.
- В результате работы был получен полный набор тестов, выявлена и исправлена ошибка, что подтверждает практическую ценность модульного тестирования для обеспечения качества кода.

10. ПРИЛОЖЕНИЕ

Исходный код программы app.py:

```
File Edit Format Run Options Window Help
def celsius_to_fahrenheit(c):
    return (c * 9/5) + 32

def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9

def meters_to_kilometers(m):
    return m / 1000

def kilograms_to_grams(kg):
    return kg * 1000

def grams_to_kilograms(g):
    return g * 1000

def main():
    while True:
        print("\n=== Конвертер единиц ===")
        print("1. Цельсий → Фаренгейт")
        print("2. Фаренгейт → Цельсий")
        print("3. Метры → Километры")
        print("4. Килограммы → Граммы")
        print("5. Граммы → Килограммы (с ошибкой)")
        print("0. Выход")

        choice = input("Выберите операцию: ")

        if choice == "0":
            print("Выход из программы.")
            break

        try:
            value = float(input("Введите значение: "))
        except ValueError:
            print("Ошибка: нужно ввести число.")
            continue

        if choice == "1":
            print(f"Результат: {celsius_to_fahrenheit(value)} °F")
        elif choice == "2":
            print(f"Результат: {fahrenheit_to_celsius(value)} °C")
        elif choice == "3":
            print(f"Результат: {meters_to_kilometers(value)} км")
        elif choice == "4":
            print(f"Результат: {kilograms_to_grams(value)} г")
        elif choice == "5":
            print(f"Результат: {grams_to_kilograms(value)} кг")
        else:
            print("Неверный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()
```

Рисунок 1 - конвертер единиц код

```

File Edit Format Run Options Window Help
def calculate_bmi(weight, height_cm):
    height_m = height_cm / 100
    try:
        bmi = weight / (height_m ** 2)
        return bmi
    except ZeroDivisionError:
        return None

def get_bmi_category(bmi):
    if bmi is None:
        return "Ошибка: рост не может быть равен нулю"
    elif bmi < 18.5:
        return "Недостаточный вес"
    elif 18.5 <= bmi <= 24.9:
        return "Нормальный вес"
    elif 25 <= bmi <= 29.9:
        return "Избыточный вес"
    else:
        return "Ожирение"

def calculate_ideal_weight(height_cm, target_bmi=22):
    height_m = height_cm / 100
    return round(target_bmi * (height_m ** 2), 2)

def save_results_to_file(weight, height_cm, bmi, category, filename="bmi_results.txt"):
    with open(filename, "a", encoding="utf-8") as f:
        f.write(f"Вес: {weight} кг, Рост: {height_cm} см, "
                f"BMI: {bmi:.2f}, Категория: {category}\n")

def load_previous_results(filename="bmi_results.txt"):
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return f.readlines()
    except FileNotFoundError:
        return ["История пока пуста.\n"]

```

Рисунок 2 - программа другого участника команды

```

File Edit Format Run Options Window Help
def compare_bmi(bmi1, bmi2):
    if bmi1 is None or bmi2 is None:
        return "Ошибка: невозможно сравнить значения"
    diff = round(bmi2 - bmi1, 2)
    if diff > 0:
        return f"Ваш BMI увеличился на {diff}"
    elif diff < 0:
        return f"Ваш BMI снизился на {abs(diff)}"
    else:
        return "Ваш BMI не изменился"

def main():
    print("=== Калькулятор BMI ===")
    weight = float(input("Введите вес (кг): "))
    height_cm = float(input("Введите рост (в сантиметрах): "))

    if weight < 0:
        print("Ошибка: вес не может быть отрицательным!")
        return
    if height_cm <= 0:
        print("Ошибка: рост не может быть отрицательным или равным нулю!")
        return

    bmi = calculate_bmi(weight, height_cm)
    category = get_bmi_category(bmi)

    print(f"Ваш BMI: {bmi:.2f}")
    print(f"Категория: {category}")

    ideal_weight = calculate_ideal_weight(height_cm)
    print(f"Идеальный вес для вашего роста: {ideal_weight} кг")

    save_results_to_file(weight, height_cm, bmi, category)

    print("\n--- История расчетов ---")
    for line in load_previous_results():
        print(line.strip())

if __name__ == "__main__":
    main()

```

Рисунок 3 - программа другого участника команды

```
*test.py - C:/Users/kuzmi/Desktop/test.py (3.13.0)*
File Edit Format Run Options Window Help

import pytest
from bmi_calculator import (
    calculate_bmi,
    get_bmi_category,
    calculate_ideal_weight,
    compare_bmi
)

# --- Тесты для calculate_bmi ---
def test_calculate_bmi_normal():
    assert round(calculate_bmi(70, 175), 2) == 22.86

def test_calculate_bmi_zero_height():
    assert calculate_bmi(70, 0) is None

def test_calculate_bmi_negative_weight():
    assert round(calculate_bmi(-70, 175), 2) == -22.86

# --- Тесты для get_bmi_category ---
def test_get_bmi_category_underweight():
    assert get_bmi_category(17) == "Недостаточный вес"

def test_get_bmi_category_normal():
    assert get_bmi_category(22) == "Нормальный вес"

def test_get_bmi_category_overweight():
    assert get_bmi_category(27) == "Избыточный вес"

def test_get_bmi_category_obesity():
    assert get_bmi_category(32) == "Ожирение"

def test_get_bmi_category_none():
    assert get_bmi_category(None) == "Ошибка: рост не может быть равен нулю"

# --- Тесты для calculate_ideal_weight ---
def test_calculate_ideal_weight_default():
    assert calculate_ideal_weight(175) == 67.38

def test_calculate_ideal_weight_custom_bmi():
    assert calculate_ideal_weight(175, target_bmi=25) == 76.56

# --- Тесты для compare_bmi ---
def test_compare_bmi_increase():
    assert compare_bmi(20, 22) == "Ваш BMI увеличился на 2"

def test_compare_bmi_decrease():
    assert compare_bmi(25, 23) == "Ваш BMI снизился на 2"

def test_compare_bmi_no_change():
    assert compare_bmi(22, 22) == "Ваш BMI не изменился"

def test_compare_bmi_with_none():
    assert compare_bmi(None, 22) == "Ошибка: невозможно сравнить"

def test_calculate_bmi_zero_weight():
    result = calculate_bmi(0, 175)
    assert result is None, f"Ожидалась ошибка при весе 0, но получено {result}"
```

Рисунок 4 - код с юнит тестами

```
platform win32 -- Python 3.13.9, pytest-8.4.0, pluggy-1.6.0 -- C:\Users\kuzmi\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13
_qbz5n2kfra8g0\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kuzmi\Desktop
plugins: anyio-4.9.0
collected 13 items

test_bmi.py::test_calculate_bmi_normal PASSED [ 7%]
test_bmi.py::test_calculate_bmi_zero_height PASSED [ 15%]
test_bmi.py::test_calculate_bmi_negative_weight PASSED [ 23%]
test_bmi.py::test_get_bmi_category_underweight PASSED [ 30%]
test_bmi.py::test_get_bmi_category_normal PASSED [ 38%]
test_bmi.py::test_get_bmi_category_overweight PASSED [ 46%]
test_bmi.py::test_get_bmi_category_obesity PASSED [ 53%]
test_bmi.py::test_get_bmi_category_none PASSED [ 61%]
test_bmi.py::test_calculate_ideal_weight_default PASSED [ 69%]
test_bmi.py::test_calculate_bmi_zero_weight FAILED [ 76%]
test_bmi.py::test_calculate_ideal_weight_custom_bmi PASSED [ 84%]
test_bmi.py::test_compare_bmi_no_change PASSED [ 92%]
test_bmi.py::test_compare_bmi_with_none PASSED [100%]

===== FAILURES =====
test_calculate_bmi_zero_weight
def test_calculate_bmi_zero_weight():
    result = calculate_bmi(0, 175)
> assert result is None, f"Ожидалась ошибка при весе 0, но получено {result}"
E   AssertionError: Ожидалась ошибка при весе 0, но получено 0.0
E   assert 0.0 is None

test_bmi.py:44: AssertionError

===== short test summary info =====
FAILED test_bmi.py::test_calculate_bmi_zero_weight - AssertionError: Ожидалась ошибка при весе 0, но получено 0.0
1 failed, 12 passed in 1.35s
```

Рисунок 5 - Результат юнит тестов